



BITS Pilani

DEEP NEURAL NETWORK DNN QUICK REFERENCE HANDBOOK

Seetha Parameswaran
WILPD - CSIS

ACTIVATION FUNCTIONS



Step: $\hat{y} = \mathbf{1}[z \geq 0]$ (1)

Identity: $f(z) = z$ (2)

Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$ (3)

Tanh: $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$ (4)

ReLU: $f(z) = \max(0, z)$ (5)

Leaky ReLU: $f(z) = \begin{cases} z & z > 0 \\ \alpha z & z \leq 0 \end{cases}, \alpha \ll 1$ (6)

GELU: $f(z) = 0.5z \left(1 + \tanh \left[\sqrt{\frac{2}{\pi}} (z + 0.044715z^3) \right] \right)$ (7)

Softmax: $\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$ (8)

LOSS FUNCTIONS



$$\text{MSE: } \mathcal{L} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 \quad (9)$$

$$\text{MAE: } \mathcal{L} = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i| \quad (10)$$

$$\text{Binary CE: } \mathcal{L} = -\frac{1}{n} \sum_i [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)] \quad (11)$$

$$\text{Categorical CE: } \mathcal{L} = -\frac{1}{n} \sum_i \sum_{k=1}^K y_{ik} \log \hat{y}_{ik} \quad (12)$$

$$\text{Hinge: } \mathcal{L} = \frac{1}{n} \sum_i \max(0, 1 - y_i \hat{y}_i) \quad (13)$$

$$\text{Perplexity: } PP = \exp \left(-\frac{1}{T} \sum_{t=1}^T \log P(w_t | w_{<t}) \right) \quad (14)$$

M2: PERCEPTRON

innovate

achieve

lead

1. Data & Initialization

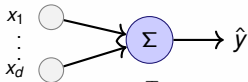
$$\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}$$

$$y \in \{0, 1\}$$

$$\mathbf{w}^{(0)} = \mathbf{0}, b^{(0)} = 0$$

forward

2. Model



$$z = \mathbf{w}^T \mathbf{x} + b$$

Step Activation

compare

Perceptron

iterate

4. Learning

$$\mathbf{w} \leftarrow \mathbf{w} + \eta(y - \hat{y})\mathbf{x}$$

$$b \leftarrow b + \eta(y - \hat{y})$$

Update only on error

3. Objective

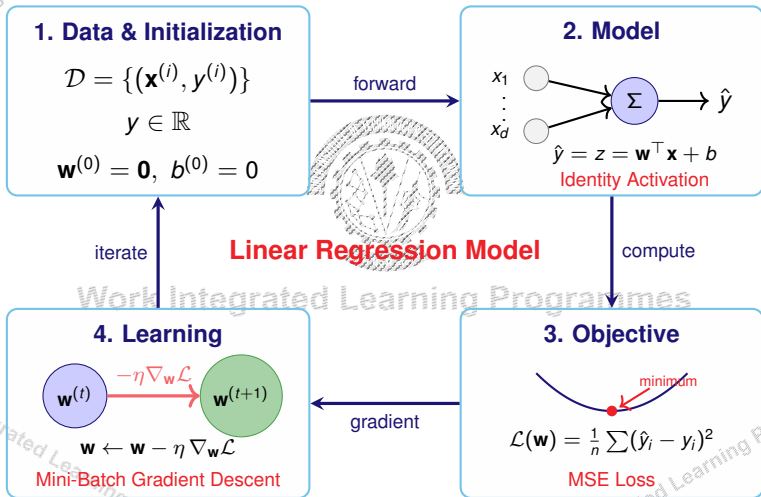
Misclassification error

$$\hat{y} = \mathbf{1}[\mathbf{w}^T \mathbf{x} + b \geq 0]$$

Correct or wrong?

error

M3: LINEAR REGRESSION



M4: BINARY CLASSIFICATION

innovate

achieve

lead

1. Data & Initialization

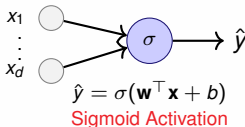
$$\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}$$

$$y \in \{0, 1\}$$

$$\mathbf{w}^{(0)} = \mathbf{0}, b^{(0)} = 0$$

forward

2. Model

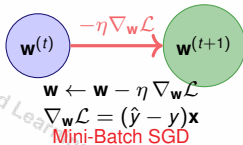


iterate

Binary Classification

compute

4. Learning



3. Objective

$$\mathcal{L} = -\frac{1}{n} \sum_i [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

Binary Cross-Entropy

gradient

M4: MULTICLASS CLASSIFICATION

innovate

achieve

lead

1. Data & Initialization

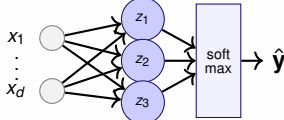
$$\mathcal{D} = \{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}$$

$$y \in \{1, \dots, K\}$$

$$\mathbf{w}^{(0)} = \mathbf{0}, b^{(0)} = 0$$

forward

2. Model



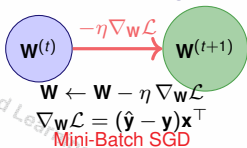
Softmax Activation

compute

Multiclass Classification

iterate

4. Learning



3. Objective

$$\mathcal{L} = -\frac{1}{n} \sum_i \sum_{k=1}^K y_{ik} \log \hat{y}_{ik}$$

Categorical Cross-Entropy

gradient

M5: DEEP FEEDFORWARD NETWORKS

innovate

achieve

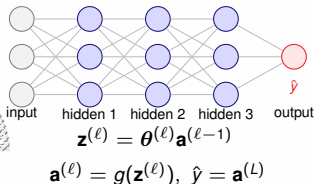
lead

1. Data & Initialization

$$\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}$$
$$y \in \mathbb{R} \text{ or } y \in \{0, 1\}$$
$$\theta \sim \text{Xavier or He}$$

forward

2. Model

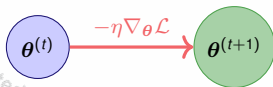


compute loss

DFNN

iterate

4. Learning



$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

Mini-Batch SGD / Adam

backprop

3. Objective

Loss = MSE or CE

$$\delta^{(L)} = \nabla_{\mathbf{a}} \mathcal{L} \odot g'(\mathbf{z}^{(L)})$$

$$\delta^{(\ell)} = (\boldsymbol{\theta}^{(\ell+1)})^{\top} \delta^{(\ell+1)} \odot g'(\mathbf{z}^{(\ell)})$$

Backpropagation

M5: DEEP FEEDFORWARD NETWORKS

innovate

achieve

lead

Training Phase:

$$\text{Forward: } \mathbf{z}^{(\ell)} = \boldsymbol{\theta}^{(\ell)} \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)} \quad (15)$$

$$\mathbf{a}^{(\ell)} = g(\mathbf{z}^{(\ell)}) \quad (16)$$

$$\text{Predict: } \hat{y} = \mathbf{a}^{(L)} \quad (17)$$

$$\text{Loss: } \mathcal{L} = \text{MSE}(\hat{y}, y) \quad \text{or} \quad \text{CE}(\hat{y}, y) \quad (18)$$

$$\text{Backprop: } \delta^{(L)} = \nabla_{\mathbf{a}} \mathcal{L} \odot g'(\mathbf{z}^{(L)}) \quad (19)$$

$$\text{Backprop: } \delta^{(\ell)} = (\boldsymbol{\theta}^{(\ell+1)})^\top \delta^{(\ell+1)} \odot g'(\mathbf{z}^{(\ell)}) \quad (20)$$

$$\text{Update: } \boldsymbol{\theta}^{(\ell)} \leftarrow \boldsymbol{\theta}^{(\ell)} - \eta \delta^{(\ell)} (\mathbf{a}^{(\ell-1)})^\top \quad (21)$$

$$\text{Parameters: } P = \sum_{\ell=1}^L n_{\ell} (n_{\ell-1} + 1) \quad (22)$$

Inference: forward pass only — no gradients

M6: CONVOLUTIONAL NEURAL NETWORKS

innovate

achieve

lead

1. Data & Initialization

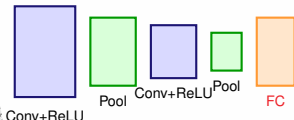
Images $\in \mathbb{R}^{H \times W \times C}$

Normalised, augmented

$$\theta \sim \mathcal{N}(0, \frac{2}{n_{in}}) \text{ (He)}$$

forward

2. Model



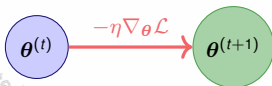
$$O(i, j) = \sum_{m, n} I(i+m, j+n) \cdot K(m, n) + b$$

CNN

predict

iterate

4. Learning



$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

Adam / SGD+Momentum

backprop

3. Objective

Loss = CE (classification)

$$\mathcal{L} = - \sum_k y_k \log \hat{y}_k$$

$$\frac{\partial \mathcal{L}}{\partial K(m, n)} = \sum_{i, j} \frac{\partial \mathcal{L}}{\partial O(i, j)} \cdot I(i+m, j+n)$$

Backprop through Conv

M6: CONVOLUTIONAL NEURAL NETWORKS

innovate

achieve

lead

Training Phase:

$$\text{Forward: } O(i, j) = \sum_m \sum_n I(i+m, j+n) \cdot K(m, n) + b \quad (23)$$

$$\text{Predict: } \hat{y} = \text{softmax}(\mathbf{W}_{\text{fc}} \mathbf{a}_{\text{flat}} + \mathbf{b}_{\text{fc}}) \quad (24)$$

$$\text{Loss: } \mathcal{L} = \text{loss}(\hat{y}, y) \quad (25)$$

$$\text{Backprop FC: } \frac{\partial \mathcal{L}}{\partial \mathbf{W}_{\text{fc}}} = \delta_{\text{fc}} \mathbf{a}_{\text{flat}}^T \quad (26)$$

$$\text{Backprop Conv: } \frac{\partial \mathcal{L}}{\partial K(m, n)} = \sum_i \sum_j \frac{\partial \mathcal{L}}{\partial O(i, j)} \cdot I(i+m, j+n) \quad (27)$$

$$\text{Update: } \theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L} \quad (28)$$

Inference: forward pass only — no gradients

M6: CNN — OUTPUT SIZES & PARAMETERS

innovate

achieve

lead

Output Sizes:

Conv out:
$$o_{\text{conv}} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1 \quad (29)$$

Pool out:
$$o_{\text{pool}} = \left\lfloor \frac{n - k_p}{s_p} \right\rfloor + 1 \quad (30)$$

Receptive field:
$$r_\ell = r_{\ell-1} + (k_\ell - 1) \prod_{i=1}^{\ell-1} s_i \quad (31)$$

Parameters:

Conv layer:
$$P_{\text{conv}} = (k^2 \cdot C_{\text{in}} + 1) \cdot C_{\text{out}} \quad (32)$$

FC layer:
$$P_{\text{fc}} = n_{\text{in}} \cdot n_{\text{out}} + n_{\text{out}} \quad (33)$$

Total:
$$P = \sum_{\text{conv}} P_{\text{conv}} + \sum_{\text{fc}} P_{\text{fc}} \quad (34)$$

M7: RECURRENT NEURAL NETWORKS

innovate

achieve

lead

1. Data & Initialization

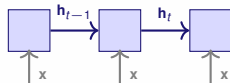
Sequences $(\mathbf{x}_1, \dots, \mathbf{x}_T)$

$$\mathbf{h}_0 = \mathbf{0}$$

$$\theta \sim \mathcal{N}(0, \frac{2}{n_{in} + n_{out}})$$

unroll

2. Model



$$\mathbf{h}_t = \tanh(W_{hh}\mathbf{h}_{t-1} + W_{hx}\mathbf{x}_t + \mathbf{b})$$

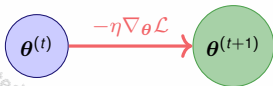
Vanilla RNN (LSTM/GRU add gates)

sum losses

iterate

RNN / LSTM / GRU

4. Learning



$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

Adam + gradient clipping

BPTT

3. Objective

Loss = CE / Perplexity

$$\mathcal{L} = \sum_{t=1}^T \mathcal{L}_t$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} = \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} + W_{hh}^T \text{diag}(1 - \mathbf{h}_{t+1}^2) \frac{\partial \mathcal{L}}{\partial \mathbf{h}_{t+1}}$$

BPTT

M7: VANILLA RNN EQUATIONS



Training Phase:

Forward: $\mathbf{h}_t = \tanh(W_{hh}\mathbf{h}_{t-1} + W_{hx}\mathbf{x}_t + \mathbf{b}_h)$ (35)

Predict: $\hat{y}_t = \text{softmax}(W_o\mathbf{h}_t + \mathbf{b}_o)$ (36)

Loss: $\mathcal{L} = \sum_{t=1}^T \mathcal{L}_t$ (37)

BPTT: $\frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} = \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} + W_{hh}^T \text{diag}(1 - \mathbf{h}_{t+1}^2) \frac{\partial \mathcal{L}}{\partial \mathbf{h}_{t+1}}$ (38)

Update: $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$ (39)

Inference: $\hat{y}_t = \text{softmax}(W_o\mathbf{h}_t + \mathbf{b}_o)$

Parameters: d = input size, h = hidden size, V = vocab size

RNN Params: $P = h(h + d + 1) + V(h + 1)$ (40)

M7: GRU / LSTM EQUATIONS



GRU:

$$\text{Update gate: } \mathbf{z}_t = \sigma(W_z \mathbf{x}_t + U_z \mathbf{h}_{t-1}) \quad (41)$$

$$\text{Reset gate: } \mathbf{r}_t = \sigma(W_r \mathbf{x}_t + U_r \mathbf{h}_{t-1}) \quad (42)$$

$$\text{Candidate: } \tilde{\mathbf{h}}_t = \tanh(W_h \mathbf{x}_t + U_h(\mathbf{r}_t \odot \mathbf{h}_{t-1})) \quad (43)$$

$$\text{Hidden: } \mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot \tilde{\mathbf{h}}_t \quad (44)$$

$$\text{GRU Params: } P = 3h(h + d + 1) + V(h + 1) \quad (45)$$

LSTM:

$$\mathbf{f}_t, \mathbf{i}_t, \mathbf{o}_t \text{ gates: } \mathbf{f}_t, \mathbf{i}_t, \mathbf{o}_t = \sigma(W_* \mathbf{x}_t + U_* \mathbf{h}_{t-1} + \mathbf{b}_*) \quad (46)$$

$$\text{Cell candidate: } \tilde{\mathbf{c}}_t = \tanh(W_c \mathbf{x}_t + U_c \mathbf{h}_{t-1}) \quad (47)$$

$$\text{Cell state: } \mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad (48)$$

$$\text{Hidden: } \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad (49)$$

$$\text{LSTM Params: } P = 4h(h + d + 1) + V(h + 1) \quad (50)$$

M8: ATTENTION MECHANISMS

innovate

achieve

lead

1. Data & Initialization

Source / target sequences

Encoder hidden states $\{h_j\}$

$$\theta \sim \mathcal{N}(0, \frac{2}{n_{in} + n_{out}})$$

encode

2. Model



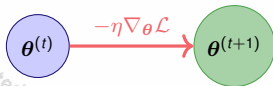
$$\alpha_{ij} = \text{softmax}(e_{ij}), \quad c_i = \sum_j \alpha_{ij} h_j$$

Attention

decode

iterate

4. Learning



$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

Adam

gradient

3. Objective

Loss = CE (Seq2Seq)

$$\mathcal{L} = - \sum_t \log P(y_t | y_{<t}, c_t)$$

Backprop through α_{ij}

Differentiable attention weights

M8: ATTENTION MECHANISMS



Additive Attention (Bahdanau):

$$\text{Score: } e_{ij} = \mathbf{v}^\top \tanh(W_s \mathbf{s}_{i-1} + W_h \mathbf{h}_j) \quad (51)$$

$$\text{Weights: } \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_k \exp(e_{ik})} \quad (52)$$

$$\text{Context: } \mathbf{c}_i = \sum_j \alpha_{ij} \mathbf{h}_j \quad (53)$$

Scaled Dot-Product Attention:

$$\text{Projections: } Q = XW^Q, \quad K = XW^K, \quad V = XW^V \quad (54)$$

$$\text{Attention: } \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (55)$$

Multi-Head Attention:

$$\text{Single head: } \text{head}_i = \text{Attention}(XW_i^Q, XW_i^K, XW_i^V) \quad (56)$$

$$\text{MHA: } \text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_H) W^O \quad (57)$$

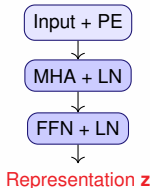
M9: TRANSFORMER VARIANTS

innovate

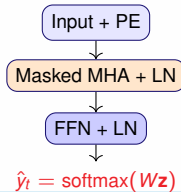
achieve

lead

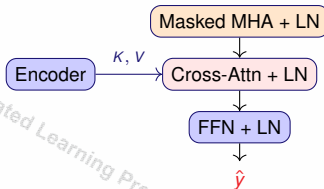
Encoder Only (e.g. BERT)



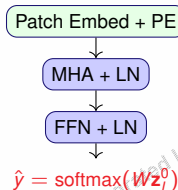
Decoder Only (e.g. GPT)



Encoder-Decoder (e.g. T5)



Vision Transformer (ViT-B/16)



M9: TRANSFORMER ENCODER



Input:

$$\text{Embedding: } \mathbf{z}_0 = \text{Embed}(X) + PE \quad (58)$$

$$PE: PE_{pos,2i} = \sin\left(\frac{pos}{10000^{2i/d}}\right), PE_{pos,2i+1} = \cos\left(\frac{pos}{10000^{2i/d}}\right) \quad (59)$$

Encoder Block ($\times N$):

$$\text{MHA + LN: } \mathbf{z}_1 = \text{LN}(\mathbf{z}_0 + \text{MHA}(\mathbf{z}_0)) \quad (60)$$

$$\text{FFN: } \text{FFN}(\mathbf{z}) = \max(0, \mathbf{z}W_1 + \mathbf{b}_1)W_2 + \mathbf{b}_2 \quad (61)$$

$$\text{FFN + LN: } \mathbf{z}_2 = \text{LN}(\mathbf{z}_1 + \text{FFN}(\mathbf{z}_1)) \quad (62)$$

Predict / Loss / Update:

$$\text{Predict: } \hat{y} = \text{softmax}(W_{\text{head}} \mathbf{z}_2) \quad (63)$$

$$\text{Loss: } \mathcal{L} = \text{loss}(\hat{y}, y) \quad (64)$$

$$\text{Update: } \theta \leftarrow \theta - \eta_t \nabla_{\theta} \mathcal{L} \quad (65)$$

M9: TRANSFORMER DECODER



Decoder Block ($\times N$):

$$\text{Masked MHA + LN: } \mathbf{z}_1 = \text{LN}(\mathbf{z}_0 + \text{MHA}_{\text{self}}(\mathbf{z}_0)) \quad (66)$$

$$\text{Cross MHA + LN: } \mathbf{z}_2 = \text{LN}(\mathbf{z}_1 + \text{MHA}_{\text{cross}}(\mathbf{z}_1, \mathbf{z}_2^{\text{enc}})) \quad (67)$$

$$\text{FFN + LN: } \mathbf{z}_3 = \text{LN}(\mathbf{z}_2 + \text{FFN}(\mathbf{z}_2)) \quad (68)$$

Cross-Attention (Q from decoder, K, V from encoder):

$$Q: Q = \mathbf{z}_1 W^Q \quad (69)$$

$$K, V: K, V = \mathbf{z}_2^{\text{enc}} W^K, \mathbf{z}_2^{\text{enc}} W^V \quad (70)$$

$$\text{Cross-Attn: } \text{MHA}_{\text{cross}} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (71)$$

Output:

$$\text{Predict: } \hat{y}_t = \text{softmax}(W_{\text{head}} \mathbf{z}_3) \quad (72)$$

M9: TRANSFORMER PARAMETERS & ViT

innovate

achieve

lead

Parameters (d = model dim, d_{ff} = FFN dim, V = vocab, N = layers):

$$\text{MHA: } P_{\text{mha}} = 4d^2 \quad (73)$$

$$\text{FFN: } P_{\text{ffn}} = 2d d_{ff} + d_{ff} + d \quad (74)$$

$$\text{Encoder (N): } P_E = N(4d^2 + 2d d_{ff} + 5d) + V \cdot d \quad (75)$$

$$\text{Decoder (N): } P_D = N(8d^2 + 2d d_{ff} + 9d) + V \cdot d \quad (76)$$

Vision Transformer (ViT):

$$\text{Patches: } N = \frac{H \times W}{P^2}, \mathbf{x}_p^i \in \mathbb{R}^{P^2 \cdot C} \quad (77)$$

$$\text{Embedding: } \mathbf{z}_0 = [\mathbf{x}_{\text{cls}}; \mathbf{x}_p^1 E; \mathbf{x}_p^2 E; \dots; \mathbf{x}_p^N E] + E_{\text{pos}} \quad (78)$$

$$\text{Patch proj.: } E \in \mathbb{R}^{(P^2 \cdot C) \times d} \quad (79)$$

$$\text{Classify: } \hat{y} = \text{softmax}(W_{\text{head}} \text{LN}(\mathbf{z}_L^0)) \quad (80)$$

$H \times W$: image size, $P \times P$: patch size, C : channels, $\mathbf{z}_L^0$: class token at layer L .

M10: OPTIMIZATION ALGORITHMS

innovate

achieve

lead

$$\text{Batch GD: } \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \frac{1}{n} \sum_i \nabla_{\boldsymbol{\theta}} \mathcal{L}_i \quad (81)$$

$$\text{SGD: } \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}_i \quad (82)$$

$$\text{Mini-batch SGD: } \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_{\boldsymbol{\theta}} \mathcal{L}_i \quad (83)$$

$$\text{Momentum: } \mathbf{v}_t = \beta \mathbf{v}_{t-1} + \nabla_{\boldsymbol{\theta}} \mathcal{L} \quad (84)$$

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \mathbf{v}_t \quad (85)$$

$$\text{NAG: } \mathbf{v}_t = \beta \mathbf{v}_{t-1} + \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta} - \beta \mathbf{v}_{t-1}) \quad (86)$$

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \mathbf{v}_t \quad (87)$$

$$\text{AdaGrad: } G_t = G_{t-1} + g_t^2 \quad (88)$$

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \frac{\eta}{\sqrt{G_t + \epsilon}} g_t \quad (89)$$

$$\text{RMSprop: } s_t = \gamma s_{t-1} + (1 - \gamma) g_t^2 \quad (90)$$

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \frac{\eta}{\sqrt{s_t + \epsilon}} g_t \quad (91)$$

M10: ADAM & LEARNING RATE SCHEDULES

innovate

achieve

lead

Adam ($\beta_1=0.9$, $\beta_2=0.999$):

$$\text{1st moment: } m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (92)$$

$$\text{2nd moment: } v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (93)$$

$$\text{Bias corr.: } \hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (94)$$

$$\text{Update: } \theta \leftarrow \theta - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t \quad (95)$$

Learning Rate Schedules:

$$\text{Step decay: } \eta_t = \eta_0 \cdot \gamma^{\lfloor t/s \rfloor} \quad (96)$$

$$\text{Exponential: } \eta_t = \eta_0 \cdot e^{-\lambda t} \quad (97)$$

$$\text{Cosine annealing: } \eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \frac{\pi t}{T}\right) \quad (98)$$

$$\text{Warmup: } \eta_t = d^{-0.5} \min(t^{-0.5}, t \cdot T_w^{-1.5}) \quad (99)$$

M11: REGULARIZATION

innovate

achieve

lead

$$\text{L2 Penalty: } \tilde{\mathcal{L}} = \mathcal{L} + \frac{\lambda}{2} \|\boldsymbol{\theta}\|^2 \quad (100)$$

$$\text{L1 Penalty: } \tilde{\mathcal{L}} = \mathcal{L} + \lambda \|\boldsymbol{\theta}\|_1 \quad (101)$$

$$\text{Dropout (train): } \tilde{\mathbf{a}}^{(\ell)} = \mathbf{m} \odot \mathbf{a}^{(\ell)}, \quad m_j \sim \text{Bernoulli}(p) \quad (102)$$

$$\text{Dropout (test): } \mathbf{a}_{\text{test}}^{(\ell)} = p \mathbf{a}^{(\ell)} \quad (103)$$

$$\text{Batch Norm: } \hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta \quad (104)$$

$$\text{Layer Norm: } \hat{x}_i = \frac{x_i - \mu_{\text{row}}}{\sqrt{\sigma_{\text{row}}^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta \quad (105)$$

PERFORMANCE METRICS - MODEL



Classification:

$$\text{Accuracy: } \text{Acc} = \frac{TP + TN}{TP + TN + FP + FN} \quad (106)$$

$$\text{Precision: } P = \frac{TP}{TP + FP} \quad (107)$$

$$\text{Recall: } R = \frac{TP}{TP + FN} \quad (108)$$

$$\text{F1: } F_1 = \frac{2PR}{P + R} \quad (109)$$

Regression:

$$\text{RMSE: } \text{RMSE} = \sqrt{\frac{1}{n} \sum_i (\hat{y}_i - y_i)^2} \quad (110)$$

$$R^2: R^2 = 1 - \frac{\sum_i (\hat{y}_i - y_i)^2}{\sum_i (\bar{y} - y_i)^2} \quad (111)$$

Deployment:

$$\text{Latency: Latency} = \frac{1}{N} \sum_{i=1}^N t_i^{\text{response}} \quad (112)$$

$$\text{Throughput: Throughput} = \frac{\text{requests processed}}{\text{second}} \quad (113)$$

$$\text{Memory (FP32): Memory} = \text{num_params} \times 4 \text{ bytes} \quad (114)$$

- **Latency:** average response time over N requests; lower is better
- **Throughput:** number of requests served per second; higher is better
- **Memory (FP32):** each parameter stored as 4-byte float
- **Memory (FP16):** halved to 2 bytes per parameter via half-precision



Thank you :)
All the best :)



Work Integrated Learning Programmes